VELOOP REWARDS - Wallet & Withdrawal Analytics Specification

Data Analyst Internship, Task 01 | Work Package D

Authenticity notice: Every field, status, category and KPI below is **Proposed** or **To Be Verified**. No fees, limits or fraud thresholds are invented, and no simulated or real VELOOP statistics appear here.

1. Purpose and design choices

Wallet and withdrawals are the highest-trust area of VELOOP's data. This specification defines a transaction model and a withdrawal lifecycle that let VELOOP explain every balance change and every payout.

- **Append-only ledger.** An entry is never edited. A reversal or adjustment is a new entry linked to the original through related_transaction_id.
- **Positive amount plus direction.** Amount is always greater than zero; credit or debit gives the sign. This avoids sign errors.
- **One unit per entry.** Different units are never added together without an agreed conversion rule (A002).
- **Idempotent writes.** A repeated request must not create a second entry (Gap G02).
- **Withdrawal is a lifecycle, not a status.** Each state has its own timestamp, so processing time and queue age can be measured.

2. Wallet transaction model (D1)

Field	Type	Required?	Definition	Validation	Status
transaction_id	string	Yes	Unique key for one wallet ledger entry	Unique; immutable; generated server-side	To Be Verified
idempotency_key	string	Optional	Key that makes a repeated request create only one entry	Unique per originating request	Proposed
user_id	string	Yes	Owner of the wallet entry	Must exist in Users	To Be Verified
created_at	timestamp (UTC)	Yes	When the entry was created	ISO 8601 UTC; not in the future	Proposed
completed_at	timestamp (UTC)	Conditional	When the entry became final	Required when status = completed; not before created_at	Proposed
transaction_type	string (enum)	Yes	Kind of entry (see transaction types)	Approved list only	Proposed
direction	string (enum)	Yes	credit (adds to balance) or debit (reduces it)	credit or debit; must agree with transaction_type	Proposed
amount	decimal	Yes	Size of the entry, stored as a positive number; direction gives the sign	Greater than 0	Proposed
unit	string (enum)	Yes	Unit of the amount (real VELOOP units only)	Approved list; never mixed within one entry	To Be Verified
status	string (enum)	Yes	State of the entry	created, completed, failed, reversed	Proposed
source_feature	string (enum)	Yes	Feature that caused the entry (task, spin, referral, withdrawal, exchange...)	Approved list	Proposed
reference_id	string	Yes	Id of the record that caused the entry (reward_id, withdrawal_id, exchange id)	Must exist in the referenced table	Proposed
related_transaction_id	string	Conditional	For reversals and adjustments, the entry being undone or changed	Required for reversal and adjustment; must exist; not equal to transaction_id	Proposed
exchange_group_id	string	Conditional	Pairs the debit and credit entries of one exchange	Required for exchange entries; exactly one debit and one credit	Proposed
balance_after	decimal	Optional	Balance in that unit after the entry	Not negative; equals previous balance plus or minus amount	To Be Verified

Transaction types

The brief names reward credit, redemption, reversal and adjustment. Types beyond what is confirmed are marked Proposed or To Be Verified.

transaction_type	Direction	Meaning	Linked To	Status
reward_credit	credit	Reward added to the wallet	reward_id	Proposed
redemption	debit	Value spent or redeemed inside VELOOP	redemption record (to be confirmed)	To Be Verified
withdrawal_debit	debit	Amount taken from wallet for a withdrawal (timing: at request or at completion is unknown)	withdrawal_id	To Be Verified
exchange_debit	debit	Unit given up in an exchange	exchange_group_id	To Be Verified
exchange_credit	credit	Unit received in an exchange	exchange_group_id	To Be Verified
reversal	credit or debit	Undoes an earlier entry; the original entry is never edited	related_transaction_id	Proposed
adjustment	credit or debit	Manual or system correction with a reason	related_transaction_id, reason_category	Proposed

3. Withdrawal lifecycle (D2)

Status	Meaning	Terminal?	Timestamp	Event	Status of This Design
requested	User has asked to withdraw	No	requested_at	withdrawal_requested	Proposed
processing	Under review or being paid out	No	processing_at	withdrawal_processing	Proposed
completed	Payout finished	Almost (can still be reversed)	final_at	withdrawal_completed	Proposed
rejected	Request refused	Yes	final_at	withdrawal_rejected	Proposed
reversed	A completed payout was undone or returned	Yes	reversed_at	withdrawal_reversed	Proposed
cancelled (name to be agreed)	User withdrew the request before processing, if this is allowed	Yes	final_at	withdrawal_cancelled	To Be Verified

Withdrawal fields

Field	Type	Required?	Definition	Validation	Status
withdrawal_id	string	Yes	Unique key for one withdrawal request	Unique; immutable	To Be Verified
user_id	string	Yes	User who requested	Must exist in Users	To Be Verified
requested_amount	decimal	Yes	Amount requested	Greater than 0	To Be Verified
requested_unit	string (enum)	Yes	Unit of the requested amount	Approved list	To Be Verified
method	string (enum)	Yes	Payout method chosen (values from real VELOOP methods only)	Approved list	To Be Verified
requested_at	timestamp (UTC)	Yes	When the request was made	ISO 8601 UTC; not in the future	Proposed
processing_at	timestamp (UTC)	Conditional	When review or processing began	Not before requested_at; required if status has been processing	Proposed
final_at	timestamp (UTC)	Conditional	When the request reached completed, rejected or reversed	Required for terminal statuses; not before requested_at	Proposed
status	string (enum)	Yes	Current state	requested, processing, completed, rejected, reversed	Proposed
reason_category	string (enum)	Conditional	Category of reason for rejection or reversal (see reason categories)	Required when status is rejected or reversed; no free text	Proposed
completed_amount	decimal	Conditional	Amount actually paid out	Required when completed; not above requested_amount unless a fee rule exists (to be verified)	To Be Verified
wallet_transaction_id	string	Conditional	Wallet entry created for this withdrawal	Must exist in Wallet; exactly one active debit per withdrawal	Proposed
reviewer_type	string (enum)	Optional	automated or manual handling	automated, manual, unknown	To Be Verified
payout_provider_reference_id	string	Optional	Payout partner's own reference, if an external provider is used	Unique per provider	To Be Verified
reversed_at	timestamp (UTC)	Conditional	When a completed withdrawal was reversed	Required when status = reversed; not before final_at	Proposed

Allowed and disallowed transitions

Disallowed transitions become data quality checks in the later quality framework.

From	To	Allowed?	Note
requested	processing	Yes	Normal path
requested	rejected	Yes	Rejected before processing starts
requested	cancelled	Yes, if cancellation exists	To be verified
processing	completed	Yes	Normal path
processing	rejected	Yes	Refused during review
completed	reversed	Yes	Original completion is kept; reversal is a later event
completed	processing	No	Impossible transition; data quality error
completed	rejected	No	Impossible transition; use reversed instead
rejected	completed	No	A new request must be a new withdrawal_id
reversed	completed	No	Impossible transition; data quality error

Reason categories

Categories only. No internal security rules or fraud thresholds are recorded or invented.

reason_category	Used For	Note	Status
insufficient_balance	Rejected	Balance below requested amount at processing time	Proposed
information_incomplete	Rejected	Payout details missing or unusable	Proposed
method_unavailable	Rejected	Chosen method not available	Proposed
policy_review	Rejected, reversed	Held back after an internal review; category only, no internal rules recorded	Proposed
provider_failure	Rejected, reversed	External payout provider failed or returned the payment	Proposed
user_request	Rejected, reversed	Cancelled or returned at the user's request	Proposed
operational_error	Reversed	Processed by mistake and corrected	Proposed
other	Any	Use sparingly; high volume signals a poor category list	Proposed

4. Withdrawal KPIs (D3)

Each KPI follows the brief's format. Formulas are reproducible and contain no targets.

KPI Name	Definition	Formula	Data Required	Source	Frequency	Stakeholder	Purpose	Limitation	Status
Withdrawal Requests	Number of withdrawal requests made in the period	COUNT(withdrawal_id) where requested_at in period	withdrawal_id, requested_at	Withdrawals table / withdrawal_requested	Daily	Product, Operations	Demand for withdrawal	Counts requests, not unique users; one user may request many times	Proposed
Completed Withdrawals	Withdrawals that reached completed in the period	COUNT(withdrawal_id) where status = completed and final_at in period	withdrawal_id, status, final_at	Withdrawals table / withdrawal_completed	Daily	Operations, Finance	Payout output	Excludes later reversals unless netted; period is by completion date	Proposed
Pending Withdrawals	Requests not yet in a terminal status at the measurement time	COUNT(withdrawal_id) where status in (requested, processing)	withdrawal_id, status	Withdrawals table	Daily snapshot	Operations	Workload and backlog	A point-in-time number; needs a snapshot to trend	Proposed
Rejected Withdrawals	Requests that ended rejected in the period	COUNT(withdrawal_id) where status = rejected and final_at in period	withdrawal_id, status, final_at, reason_category	Withdrawals table / withdrawal_rejected	Daily	Operations, Product	Friction and issues	Reason quality depends on reason_category being filled	Proposed
Reversed Withdrawals	Completed withdrawals that were later reversed	COUNT(withdrawal_id) where status = reversed and reversed_at in period	withdrawal_id, status, reversed_at, reason_category	Withdrawals table / withdrawal_reversed	Weekly	Operations, Finance	Payout reliability	Reversals may appear long after completion	Proposed
Processing Time	Time from request to a terminal status	Median and 90th percentile of (final_at - requested_at) for terminal withdrawals	requested_at, final_at, status	Withdrawals table	Weekly	Operations, Product	Speed of service	Excludes still-pending requests, so it can look better than reality; report with pending age	Proposed
Completion Rate	Share of requests from a period that ended completed	COUNT(completed) / COUNT(requests) for requests made in the period, shown with a separate resolved-only version excluding pending	withdrawal_id, requested_at, status	Withdrawals table	Weekly	Product, Operations	Reliability of getting paid	Recent periods look low until requests resolve; always show pending share beside it	Proposed
Pending Queue Age	How long the oldest and typical pending requests have waited	Median and maximum of (now - requested_at) for pending requests	withdrawal_id, requested_at, status	Withdrawals table	Daily	Operations	Spot stuck requests	Needs a defined 'now' and consistent timezone	Proposed
Withdrawal Volume	Total amount requested and completed, per unit	SUM(requested_amount) and SUM(completed_amount) grouped by requested_unit and period	requested_amount, completed_amount, requested_unit	Withdrawals table	Daily	Finance, Management	Size of payouts	Never add different units together; conversion rule needed (A002)	Proposed
Method Distribution	Share of requests and completions by method	COUNT(withdrawal_id) grouped by method / total COUNT	method, withdrawal_id, status	Withdrawals table	Weekly	Product, Operations	Which methods users choose	Depends on the real list of methods (A006)	Proposed

5. Reconciliation checks

These checks link wallet, rewards and withdrawals. They feed the data quality framework and the implementation backlog.

Check	Compares	Rule	Why It Matters	Priority
Every credited reward has a wallet entry	Rewards vs Wallet	Each reward_credited has exactly one reward_credit transaction with the same reference_id	Rewards not in the wallet, or wallet entries without a reward, break trust in balances	Critical
Wallet balance reconciles	Wallet vs balance	Per user and unit: sum of completed credits minus completed debits = reported balance	Detects missing, duplicate or edited entries	Critical
Every completed withdrawal has a debit	Withdrawals vs Wallet	Each completed withdrawal links to exactly one withdrawal_debit entry	Prevents paying without deducting, or deducting twice	Critical
Reversals point to a real original	Wallet	Each reversal has related_transaction_id that exists and is not itself a reversal	Prevents orphan or chained reversals	High
Exchanges are paired	Wallet	Each exchange_group_id has one debit and one credit	Prevents one-sided exchanges	High
No duplicate entries	Wallet	No two entries share an idempotency_key or transaction_id	Detects double crediting	Critical
Withdrawal timestamps are ordered	Withdrawals	requested_at <= processing_at <= final_at	Impossible order points to bad data	Medium

6. Open questions for verification

- Which withdrawal methods exist, and are they all manual? (A006, A007)
- Is the wallet amount held or debited at request time, or only on completion? This changes balance and pending-queue logic.
- Are there fees, limits or minimum amounts? (None are invented here.)
- In what unit are withdrawals paid, and how does it relate to reward units? (A002)
- Can a user cancel a withdrawal before processing?
- Is the wallet an append-only ledger today, or are balances edited in place?
- Does a payout provider return its own reference and status? (A009)

Next: Work Package E - Referral and Engagement Analytics.